

A. Degrees

1. `neighbors_for_person(person_id)` - This function finds all co-authors of a given scientist. It returns a set of tuples, where each tuple contains a paper ID and the ID of a co-author.
2. `person_id_for_name(name)` - This function converts a scientist's name to their unique ID. If multiple scientists have the same name, it prompts the user to select the intended scientist.
3. `main()` - The main function that:
 - Processes command-line arguments
 - Loads data from a specified directory
 - Prompts the user for source and target scientist names
 - Finds the shortest path between them using a function called `shortest_path()`
 - Displays the connection path if one exists
4. The final `if __name__ == "__main__":` block ensures the `main()` function runs when the script is executed directly.

```
import csv
import sys
from collections import deque

# Maps names to a set of corresponding scientist_ids
names = {}

# Maps scientist_ids to a dict of: name, set of paper_ids
people = {}

# Maps paper_ids to a dict of: title, year, set of author_ids
papers = {}

class Node:
    def __init__(self, state, parent, action):
        self.state = state # scientist_id
        self.parent = parent # Node
        self.action = action # paper_id

def load_data(directory):
    """
    Load data from CSV files into memory.
    """
    # Load scientists
    with open(f"{directory}/scientists.csv", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            people[row["scientist_id"]] = {
                "name": row["name"],
                "papers": set()
            }
            name_key = row["name"].lower()
            names.setdefault(name_key, set()).add(row["scientist_id"])

    # Load papers
    with open(f"{directory}/papers.csv", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            papers[row["paper_id"]] = {
                "title": row["title"],
                "year": row["year"],
                "authors": set()
            }

    # Load authorship
    with open(f"{directory}/authors.csv", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            try:
                people[row["scientist_id"]]["papers"].add(row["paper_id"])
                papers[row["paper_id"]]["authors"].add(row["scientist_id"])
            except KeyError:
                continue

def shortest_path(source, target):
    """
    Find shortest path between two scientists using BFS.
    Returns list of (paper_id, scientist_id) tuples.
    """
```

```

start = Node(source, None, None)
frontier = deque([start])
explored = set()
frontier_ids = {source}

while frontier:
    node = frontier.popleft()
    frontier_ids.remove(node.state)
    explored.add(node.state)

    for paper_id, neighbor_id in neighbors_for_person(node.state):
        if neighbor_id in explored or neighbor_id in frontier_ids:
            continue

        child = Node(neighbor_id, node, paper_id)

        if neighbor_id == target:
            path = []
            while child.parent is not None:
                path.append((child.action, child.state))
                child = child.parent
            path.reverse()
            return path

        frontier.append(child)
        frontier_ids.add(neighbor_id)

return None

def neighbors_for_person(person_id):
    """
    Returns (paper_id, person_id) pairs for co-authors of a given scientist.
    """
    neighbors = set()
    for paper_id in people[person_id]["papers"]:
        for author_id in papers[paper_id]["authors"]:
            if author_id != person_id:
                neighbors.add((paper_id, author_id))
    return neighbors

def person_id_for_name(name):
    """
    Resolves a person's name to a scientist_id.
    """
    person_ids = list(names.get(name.lower(), set()))
    if not person_ids:
        return None
    elif len(person_ids) > 1:
        print(f"Multiple scientists found for '{name}':")
        for pid in person_ids:
            print(f"{pid}: {people[pid]['name']}")
        try:
            selected = input("Intended Person ID: ").strip()
            if selected in person_ids:
                return selected
        except Exception:
            pass
        return None
    else:
        return person_ids[0]

def main():
    if len(sys.argv) != 2:
        sys.exit("Usage: python degrees.py [directory]")
    directory = sys.argv[1]

    print("Loading data...")
    load_data(directory)
    print("Data loaded.")

    source_name = input("Name: ").strip()
    source = person_id_for_name(source_name)
    if source is None:
        sys.exit(f"Scientist '{source_name}' not found.")

    target_name = input("Name: ").strip()
    target = person_id_for_name(target_name)
    if target is None:
        sys.exit(f"Scientist '{target_name}' not found.")

```

```

path = shortest_path(source, target)

if path is None:
    print("No connection found.")
else:
    print(f"{len(path)} degrees of separation.")
    current = source
    for i, (paper_id, person_id) in enumerate(path, 1):
        scientist1 = people[current]["name"]
        scientist2 = people[person_id]["name"]
        paper = papers[paper_id]["title"]
        print(f"{i}: {scientist1} and {scientist2} co-authored \"{paper}\"")
        current = person_id

if __name__ == "__main__":
    main()

```

B. Sudoku

1. The code contains methods for solving a Sudoku puzzle using constraint satisfaction techniques:

- `solve()`: Main solving method that enforces constraints and uses backtracking
- `update_board()`: Updates the Sudoku board with values from the solution domains
- `print_board()`: Displays the Sudoku board in a formatted way

2. The `load_puzzle_from_file()` function reads a Sudoku puzzle from a text file, validating that it has the correct dimensions (9x9).

3. The main execution block:

- Checks for a command-line argument (puzzle file)
- Loads the puzzle from the specified file
- Creates a Sudoku solver instance
- Attempts to solve the puzzle
- Displays the solution or an error message

This code represents the core solving functionality and program execution flow for a Sudoku solver that uses AI constraint satisfaction techniques.

import copy

class Sudoku_AI_solver: def **init**(self, board): self.board = board # 9x9 grid self.domains = self.initialize_domains()

```

def initialize_domains(self):
    domains = {}
    for i in range(9):
        for j in range(9):
            if self.board[i][j] == 0:
                domains[(i, j)] = set(range(1, 10))
            else:
                domains[(i, j)] = {self.board[i][j]}
    return domains

def enforce_node_consistency(self):
    for (i, j), domain in self.domains.items():
        if len(domain) == 1:
            self.domains[(i, j)] = set(domain)

def revise(self, x, y):
    revised = False
    if len(self.domains[y]) == 1:
        val = next(iter(self.domains[y]))
        if val in self.domains[x] and len(self.domains[x]) > 1:
            self.domains[x].remove(val)
            revised = True
    return revised

def ac3(self):
    queue = [(x, y) for x in self.domains for y in self.get_neighbors(x)]
    while queue:
        (x, y) = queue.pop(0)
        if self.revise(x, y):

```

```

        if not self.domains[x]:
            return False
        for z in self.get_neighbors(x):
            if z != y:
                queue.append((z, x))
        return True

def get_neighbors(self, cell):
    i, j = cell
    neighbors = set()

    # Row and column
    for k in range(9):
        if k != j:
            neighbors.add((i, k))
        if k != i:
            neighbors.add((k, j))

    # Box
    box_i = 3 * (i // 3)
    box_j = 3 * (j // 3)
    for x in range(box_i, box_i + 3):
        for y in range(box_j, box_j + 3):
            if (x, y) != (i, j):
                neighbors.add((x, y))

    return neighbors

def assignment_complete(self, assignment):
    return all(len(values) == 1 for values in assignment.values())

def consistent(self, assignment):
    for cell, val_set in assignment.items():
        if len(val_set) != 1:
            continue
        val = next(iter(val_set))
        for neighbor in self.get_neighbors(cell):
            if val in assignment[neighbor] and len(assignment[neighbor]) == 1:
                return False
    return True

def select_unassigned_variable(self, assignment):
    unassigned = [v for v in assignment if len(assignment[v]) > 1]
    if not unassigned:
        return None
    return min(unassigned, key=lambda x: len(assignment[x]))

def order_domain_values(self, var, assignment):
    def count_constraints(val):
        count = 0
        for neighbor in self.get_neighbors(var):
            if val in assignment[neighbor]:
                count += 1
        return count
    return sorted(assignment[var], key=count_constraints)

def backtrack(self, assignment):
    if self.assignment_complete(assignment):
        return assignment

    var = self.select_unassigned_variable(assignment)
    for value in self.order_domain_values(var, assignment):
        new_assignment = copy.deepcopy(assignment)
        new_assignment[var] = {value}

        solver = Sudoku_AI_solver(self.board)
        solver.domains = new_assignment
        if solver.consistent(new_assignment):
            result = self.backtrack(new_assignment)
            if result:

```

```

        return result
    return None

def solve(self):
    self.enforce_node_consistency()
    self.ac3()
    result = self.backtrack(copy.deepcopy(self.domains))
    if result:
        self.domains = result
        self.update_board()
    return self.board

def update_board(self):
    for (i, j), values in self.domains.items():
        if len(values) == 1:
            self.board[i][j] = next(iter(values))

def print_board(self):
    for i, row in enumerate(self.board):
        print(" ".join(str(num) if num != 0 else "." for num in row))
        if i % 3 == 2 and i != 8:
            print("-" * 21)

```

import sys

```
def load_puzzle_from_file(filename): board = [] with open(filename, "r") as f: for line in f: row = [int(x) for x in line.strip().split()] if len(row) != 9:
raise ValueError("Each row must have 9 numbers.") board.append(row) if len(board) != 9: raise ValueError("Puzzle must have 9 rows.") return
board
```

```
if name == "main": if len(sys.argv) != 2: print("Usage: python sudoku_AI_solver.py puzzle.txt") sys.exit(1)
```

```

puzzle_file = sys.argv[1]
try:
    puzzle = load_puzzle_from_file(puzzle_file)
except Exception as e:
    print(f"Error loading puzzle: {e}")
    sys.exit(1)

solver = Sudoku_AI_solver(puzzle)
solved = solver.solve()

if solved:
    print("\nPuzzle solved successfully:\n")
    solver.print_board()
else:
    print("\nNo solution could be found for this puzzle.")

```

C. Machine Learning: Traffic Sign Recognition

A function called `build_model()` that creates a convolutional neural network (CNN) for image classification. Here's a step-by-step explanation:

1. The function creates a Sequential model, which is a linear stack of layers.
2. The architecture consists of:
 - First convolutional layer with 32 filters of size 3x3, ReLU activation, and input shape matching the image dimensions
 - Max pooling layer that reduces spatial dimensions by half
 - Second convolutional layer with 64 filters of size 3x3 and ReLU activation
 - Another max pooling layer
 - Flatten layer to convert the 2D feature maps to a 1D vector
 - Dense (fully connected) layer with 128 neurons and ReLU activation
 - Dropout layer with 50% rate to prevent overfitting
 - Output layer with neurons equal to NUM_CLASSES and softmax activation for multi-class classification
3. The model is compiled with:
 - Adam optimizer
 - Categorical cross-entropy loss function (standard for multi-class classification)
 - Accuracy as the evaluation metric

4. The function returns the compiled model ready for training.

This is a standard CNN architecture commonly used for image classification tasks.

```
import os
import sys
import cv2
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical

# Constants
IMG_HEIGHT = 30
IMG_WIDTH = 30
NUM_CLASSES = 43

def load_data(data_dir):
    print("Loading data...")
    images = []
    labels = []

    for label in range(NUM_CLASSES):
        label_dir = os.path.join(data_dir, str(label))
        if not os.path.isdir(label_dir):
            continue

        for filename in os.listdir(label_dir):
            if filename.endswith(".ppm") or filename.endswith(".png") or filename.endswith(".jpg"):
                try:
                    img_path = os.path.join(label_dir, filename)
                    image = cv2.imread(img_path)
                    image = cv2.resize(image, (IMG_WIDTH, IMG_HEIGHT))
                    images.append(image)
                    labels.append(label)
                except Exception as e:
                    print(f"Failed to load {filename}: {e}")

    print("Data loaded.")
    return np.array(images), np.array(labels)

def build_model():
    model = Sequential([
        Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
        MaxPooling2D(2, 2),
        Conv2D(64, (3, 3), activation='relu'),
        MaxPooling2D(2, 2),
        Flatten(),
        Dense(128, activation='relu'),
        Dropout(0.5),
        Dense(NUM_CLASSES, activation='softmax')
    ])
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

def main():
    if len(sys.argv) < 2:
        sys.exit("Usage: python traffic_signs.py data_directory [model.h5]")

    data_dir = sys.argv[1]
    model_filename = sys.argv[2] if len(sys.argv) == 3 else None

    # Load and preprocess data
    X, y = load_data(data_dir)
    X = X.astype("float32") / 255.0 # normalize
    y_cat = to_categorical(y, NUM_CLASSES)

    # Split into train and test
    X_train, X_test, y_train, y_test = train_test_split(X, y_cat, test_size=0.2, random_state=42)

    # Train model
    print("Training model...")
    model = build_model()
    model.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.1)
```

```

# Save model if filename given
if model_filename:
    model.save(model_filename)
    print(f"Model saved to {model_filename}")

# Evaluate model
print("Evaluating model...")
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=2)
print(f"Model accuracy: {test_acc:.4f}")

# Confusion matrix
y_true = np.argmax(y_test, axis=1)
y_pred = np.argmax(model.predict(X_test), axis=1)
print("\nClassification Report:")
print(classification_report(y_true, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_true, y_pred))

if __name__ == "__main__":
    main()

```

Matplotlib is building the font cache; this may take a moment.
Loading data...
Data loaded.

ValueError Traceback (most recent call last)

```

Cell In[7], line 100
     96     print(confusion_matrix(y_true, y_pred))
     99 if __name__ == "__main__":
--> 100     main()

```

```

Cell In[7], line 73, in main()
     70 y_cat = to_categorical(y, NUM_CLASSES)
     72 # Split into train and test
--> 73 X_train, X_test, y_train, y_test = train_test_split(X, y_cat, test_size=0.2, random_state=42)
     75 # Train model
     76 print("Training model...")

```

File ~\anaconda3\Lib\site-packages\sklearn\utils_param_validation.py:213, in validate_params.<locals>.decorator.
<locals>.wrapper(*args, **kwargs)

```

    207 try:
    208     with config_context(
    209         skip_parameter_validation=(
    210             prefer_skip_nested_validation or global_skip_validation
    211         )
    212     ):
--> 213         return func(*args, **kwargs)
    214 except InvalidParameterError as e:
    215     # When the function is just a wrapper around an estimator, we allow
    216     # the function to delegate validation to the estimator, but we replace
    217     # the name of the estimator by the name of the function in the error
    218     # message to avoid confusion.
    219     msg = re.sub(
    220         r"parameter of \w+ must be",
    221         f"parameter of {func.__qualname__} must be",
    222         str(e),
    223     )

```

File ~\anaconda3\Lib\site-packages\sklearn\model_selection_split.py:2780, in train_test_split(test_size, train_size, random_state, shuffle, stratify, *arrays)

```

    2777 arrays = indexable(*arrays)
    2779 n_samples = _num_samples(arrays[0])
-> 2780 n_train, n_test = _validate_shuffle_split(
    2781     n_samples, test_size, train_size, default_test_size=0.25
    2782 )
    2784 if shuffle is False:
    2785     if stratify is not None:

```

File ~\anaconda3\Lib\site-packages\sklearn\model_selection_split.py:2410, in _validate_shuffle_split(n_samples, test_size, train_size, default_test_size)

```

    2407 n_train, n_test = int(n_train), int(n_test)
    2409 if n_train == 0:
-> 2410     raise ValueError(
    2411         "With n_samples={}, test_size={} and train_size={}, the "
    2412         "resulting train set will be empty. Adjust any of the "
    2413         "aforementioned parameters.".format(n_samples, test_size, train_size)
    2414     )
    2416 return n_train, n_test

```

ValueError: With n_samples=0, test_size=0.2 and train_size=None, the resulting train set will be empty. Adjust any of the

